



|            |                              |                                                         |
|------------|------------------------------|---------------------------------------------------------|
|            | search/                      | <- Hybrid search: BM25 + vector, Reciprocal Rank        |
| Fusion     |                              |                                                         |
|            | embeddings/                  | <- Snowflake arctic-embed-xs (384D)                     |
|            | group/                       | <- Cross-repo group management                          |
|            | wiki/                        | <- LLM-powered wiki generation                          |
|            | mcp/                         | <- MCP server: tools.ts, resources.ts, local-           |
| backend.ts |                              |                                                         |
|            | cli/                         |                                                         |
|            | gitnexus-web/                | <- Vite + React browser UI (graph explorer + AI chat)   |
|            | gitnexus-shared/             | <- Shared TypeScript types + constants                  |
|            | gitnexus-claude-plugin/      | <- Claude Code plugin metadata                          |
|            | gitnexus-cursor-integration/ | <- Cursor-specific integration                          |
|            | .claude/skills/              | <- Agent skills tự động cài khi analyze                 |
|            | gitnexus-exploring/          |                                                         |
|            | gitnexus-debugging/          |                                                         |
|            | gitnexus-impact-analysis/    |                                                         |
|            | gitnexus-plan/               | <- /gitnexus-plan slash command                         |
|            | gitnexus-work/               | <- /gitnexus-work (execute plan as atomic commits)      |
|            | gitnexus-review/             | <- /gitnexus-review (PR review backed by graph)         |
|            | gitnexus-lfg/                | <- /gitnexus-lfg (plan → gate → work → review pipeline) |
|            | eval/                        | <- Evaluation harnesses                                 |
|            | pr-swarm-review/             | <- Multi-persona PR review system                       |

## Công nghệ stack

| Layer               | Technology                                                             |
|---------------------|------------------------------------------------------------------------|
| ---                 | ---                                                                    |
| Language            | TypeScript (Node.js)                                                   |
| AST Parser          | Tree-sitter (native bindings + WASM cho browser)                       |
| Graph DB            | LadybugDB (custom graph DB tích hợp, tương tự LanceDB/Kuzu)            |
| Search              | BM25 + Snowflake arctic-embed-xs (384D vector), Reciprocal Rank Fusion |
| Community detection | Leiden algorithm                                                       |
| Protocol            | MCP (Model Context Protocol — Anthropic standard)                      |
| Web UI              | Vite + React                                                           |
| Packaging           | npm ( <a href="#">gitnexus</a> package)                                |
| CI                  | GitHub Actions (composite actions)                                     |

## 3. Phân tích kỹ thuật

### 3.1 Ingestion Pipeline — 19-phase DAG

Pipeline chạy 19 phase theo DAG (Directed Acyclic Graph) — mỗi phase khai báo `deps` tường minh, runner dùng Kahn's topological sort để validate và thực thi. Không có plugin system — graph phase là compile-time static (`runner.ts`).

Thứ tự DAG:

```
scan → structure → [springConfig, markdown, cobol] → parse → [routes, tools, orm]
→ crossFile → scopeResolution → [springAutoConfiguration, springAop]
→ pruneLocalSymbols → mro → springAopInheritance → di → communities → processes
```

Mỗi phase nhận `PipelineContext` (shared mutable `KnowledgeGraph`) và `ReadonlyMap<string, PhaseResult>` chỉ chứa các dep đã khai báo — runner filter để ngăn hidden coupling. Pattern này đảm bảo phases không thể "đọc lên" output của phase không được khai báo là dep.

Sau pipeline, graph được load vào LadybugDB qua CSV streaming để không bị OOM với repo lớn.

### 3.2 Language-Agnostic Architecture — Plugin Pattern

Một trong những điểm thiết kế nổi bật nhất: toàn bộ logic xử lý ngôn ngữ được tách biệt hoàn toàn qua 2 interface:

**LanguageProvider** — cung cấp Tree-sitter queries, import semantics, MRO strategy:

```
// gitnexus/src/core/ingestion/language-provider.ts
interface LanguageProvider {
  id: string;
  extensions: string[];
  treeSitterQueries: string; // S-expressions cho Tree-sitter
  importSemantics: 'named' | 'wildcard-leaf' | 'wildcard-transitive' | 'namespace';
  mroStrategy: 'first-wins' | 'c3' | 'ruby-mixin' | 'none';
  // ...
}
```

**ScopeResolver** — xử lý call/import resolution, MRO, inheritance:

```
// gitnexus/src/core/ingestion/scope-resolution/contract/scope-resolver.ts
interface ScopeResolver {
  languageProvider: LanguageProvider;
  populateOwners(parsed: ParsedFile): void;
  buildMro(graph, parsed, nodeLookup): Map<DefId, DefId[]>;
  resolveImportTarget(target, fromFile, allFiles): string | null;
  // ... nhiều hook tùy chọn khác
}
```

Thêm ngôn ngữ mới = implement 2 interface + đăng ký vào `SCOPE_RESOLVERS` map. Không cần sửa shared code. CI auto-discover qua TypeScript `satisfies` constraint — thiếu language là compile error.

Unified capture tags (`@definition.class`, `@definition.function`, `@call.name...`) đảm bảo downstream extraction không cần branch theo ngôn ngữ.

### 3.3 Precomputed Relational Intelligence — Điểm cốt lõi

Khác với Traditional Graph RAG yêu cầu LLM tự query nhiều lần, GitNexus precompute tất cả tại thời điểm index:

- **Communities** (phase `communities`): Leiden algorithm phát hiện nhóm symbol liên quan (functional communities). Kết quả: agent có thể hỏi "module này làm gì?" thay vì đọc hàng chục file.
- **Processes** (phase `processes`): Trace execution flows từ entry points qua call chains. Kết quả: agent biết "LoginFlow đi qua 7 bước, bước 2 là `validateUser`" — một query thay vì traverse thủ công.
- **Impact** (MCP tool `impact`): Blast radius với depth grouping và confidence score. Kết quả: `d=1` là WILL BREAK, `d=2` là LIKELY AFFECTED — không cần agent tự suy luận.

### 3.4 MCP Tools — 17 công cụ

MCP server expose 17 tools qua giao thức chuẩn (không phải REST API riêng):

| Tool                | Chức năng quan trọng                                  |
|---------------------|-------------------------------------------------------|
| ---                 | ---                                                   |
| <code>impact</code> | Blast radius — "cái gì phụ thuộc vào X?" với depth và |

|                     |                                                                 |
|---------------------|-----------------------------------------------------------------|
|                     | confidence                                                      |
| context             | 360° view của một symbol — callers, callees, processes          |
| query               | Hybrid search BM25+vector, kết quả group theo process           |
| detect_changes      | Map git diff → affected symbols và processes (pre-commit check) |
| trace               | Shortest path giữa 2 symbol (call + class-member edges)         |
| rename              | Multi-file rename với graph + text search, có dry_run           |
| cypher              | Raw Cypher query cho power users                                |
| route_map           | API route → handler → consumer mappings                         |
| shape_check         | Validate API response shape vs consumer property access         |
| explain / pdg_query | Control/data flow analysis (cần --pdg index)                    |

### 3.5 Agent Skills — Tự động cài khi analyze

gitnexus analyze tự động cài các skill files vào .claude/skills/ và .agents/skills/:

- **Standard skills:** exploring, debugging, impact-analysis, refactoring, guide, cli
- **Advanced skills:** gitnexus-plan, gitnexus-work, gitnexus-review, gitnexus-lfg
- **Repo-specific skills** (--skills flag): detect functional areas via Leiden community, generate per-area skill file mô tả key files, entry points, cross-area connections

Skill files là Markdown với description frontmatter để agent biết khi nào gọi (pattern giống với KZTEK .claude/commands/).

### 3.6 Worker Pool và Incremental Indexing

- Parse được thực hiện qua worker pool song song (không có sequential fallback, --workers 0 bị reject)
- Files chia thành chunk ~20MB để bound memory
- ParsedFile được serialize qua disk-backed store (không giữ trong RAM) — hỗ trợ repo cực lớn như Linux kernel
- Incremental: analyze detect stale bằng lastCommit == HEAD, skip nếu không đổi; detect\_changes dùng git diff để re-index phần thay đổi
- Parse cache keyed theo file content hash + schema version — warm run không spawn worker

### 3.7 Điểm mạnh / Điểm yếu

#### Điểm mạnh:

- Precomputed intelligence: agent không cần nhiều turn để gather context
- Language-agnostic plugin system: thêm ngôn ngữ không đụng shared code
- MCP standard: tích hợp được với mọi AI agent hỗ trợ MCP
- Worker pool + disk-backed store: scale tới repo cực lớn
- Incremental indexing: analyze nhanh trên daily use
- Agent skills auto-install: zero-friction adoption
- Optional CFG/PDG (--pdg): control/data flow, taint analysis cho security

#### Điểm yếu:

- License PolyForm Noncommercial: **KHÔNG được dùng cho commercial use** — đây là hạn chế nghiêm trọng nếu tính tích hợp vào sản phẩm thương mại của KZTEK
- CFG/PDG chỉ có TypeScript/JavaScript — C#, Python... chưa có
- Node.js only: không phải language runtime phổ biến nhất trong stack KZTEK (C#/.NET)
- LadybugDB là custom DB — không có community support ngoài project này
- `npm install` nặng, yêu cầu C++ toolchain cho một số grammars (Dart, Proto, Swift, Kotlin)
- Web UI giới hạn ~5k files nếu dùng browser-only mode

## 4. Hiện trạng KZTEK

**Khu vực tương ứng:** Cơ chế cung cấp context codebase cho AI agents trong workflow KZTEK.

### 4.1 Hiện trạng thực tế (đã đọc trực tiếp)

KZTEK hiện dùng `code-graph/CODE-GRAPH.md` — file Markdown tĩnh, cập nhật thủ công bởi coding agents sau mỗi thay đổi code:

Trích `code-graph/CODE-GRAPH.md` (line 1–8):

"File này được duy trì tự động bởi coding agents."

"Đọc file này TRƯỚC khi đọc source code để hiểu cấu trúc dự án mà không cần mở từng file."

"LƯU Ý QUAN TRỌNG: Đây là AI Agent Framework workspace, **KHÔNG** phải codebase sản phẩm."

Workspace hiện tại là AI agent config framework — chưa có sản phẩm code thực tế. CODE-GRAPH.md ghi nhận cấu trúc `.claude/`, `KztekComponent/`, `scripts/` nhưng không có graph thực (không có call graph, community, process trace).

### 4.2 Tooling hiện có

| Tool                                                 | Trạng thái         | Ghi chú                                                                                                                        |
|------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------|
| ---                                                  | ---                | ---                                                                                                                            |
| <code>code-graph/CODE-GRAPH.md</code>                | Tồn tại, thủ công  | Cập nhật bởi agents sau code change, không tự động                                                                             |
| <code>graphify</code> (PyPI <code>graphifyy</code> ) | Tùy chọn, chưa cài | CLI Python, không có MCP exposure, không có community detection. GOTCHAS G006: tên package là <code>graphifyy</code> (2 chữ y) |
| MCP server codebase                                  | Chưa có            | Không có MCP server nào expose graph codebase                                                                                  |
| Automated AST indexing                               | Chưa có            | Không có                                                                                                                       |
| Community/process tracing                            | Chưa có            | Không có                                                                                                                       |

### 4.3 Cơ chế hiện tại cho coding agents

Khi agent muốn hiểu codebase, theo CLAUDE.md §17.1:

1. Glob `code-graph/CODE-GRAPH.md` → Read file
2. Nếu trả lời được  $\geq 3/5$  câu hỏi → bắt đầu coding
3. Thiếu info → Read thêm source file liên quan trực tiếp

Không có query API, không có blast-radius tool, không có process trace — agent phải Grep/Read thủ công.

## 5. So sánh GitNexus vs Hiện trạng KZTEK

| Khía cạnh                  | GitNexus                                                                                                | KZTEK hiện tại                                               |
|----------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| ---                        | ---                                                                                                     | ---                                                          |
| <b>Indexing</b>            | Tự động, AST-based Tree-sitter                                                                          | Thủ công bởi coding agent sau mỗi code change                |
| <b>Storage</b>             | LadybugDB graph DB (persistent, file <code>.gitnexus/</code> )                                          | Markdown file <code>code-graph/CODE-GRAPH.md</code>          |
| <b>Query</b>               | 17 MCP tools — <code>impact</code> , <code>context</code> , <code>query</code> , <code>trace</code> ... | Grep/Read thủ công; CODE-GRAPH giải đáp top-level câu hỏi    |
| <b>Community detection</b> | Leiden algorithm tự động phát hiện functional cluster                                                   | Không có                                                     |
| <b>Process tracing</b>     | Tự động trace execution flows từ entry points                                                           | Không có                                                     |
| <b>Impact analysis</b>     | <code>impact</code> tool: blast radius 1 query, có confidence score                                     | Phải đọc/trace thủ công — không có tool                      |
| <b>Languages</b>           | 14 ngôn ngữ (C#, TS, Python, Java, Go, Rust...)                                                         | N/A (workspace này là agent config, không phải product code) |
| <b>Incremental update</b>  | git-diff based, chỉ re-parse phần thay đổi                                                              | Không có — cập nhật toàn bộ file thủ công                    |
| <b>Agent skills</b>        | Tự động cài khi <code>analyze</code> (exploring, debugging, impact...)                                  | Thủ công tạo <code>.claude/commands/*.md</code>              |
| <b>Protocol</b>            | MCP chuẩn — tương thích mọi AI agent                                                                    | Proprietary — chỉ Claude Code đọc CLAUDE.md/CODE-GRAPH.md    |
| <b>License</b>             | PolyForm Noncommercial — <b>cấm dùng thương mại</b>                                                     | N/A                                                          |

**Nhận xét tổng:** GitNexus và CODE-GRAPH.md của KZTEK giải quyết cùng vấn đề (cung cấp context codebase cho AI agent) nhưng ở quy mô và độ automation khác nhau hoàn toàn. CODE-GRAPH.md là giải pháp lightweight, phù hợp cho workspace config hiện tại. GitNexus phù hợp cho product codebase lớn (hàng nghìn file) cần real-time blast radius và cross-file tracing.

## 6. Thông tin repo

| Mục                     | Chi tiết                                                                                                                |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|
| ---                     | ---                                                                                                                     |
| <b>URL</b>              | <a href="https://github.com/abhigyanpatwari/GitNexus">https://github.com/abhigyanpatwari/GitNexus</a>                   |
| <b>License</b>          | PolyForm Noncommercial 1.0.0 — <b>KHÔNG dùng cho mục đích thương mại</b>                                                |
| <b>npm package</b>      | <code>gitnexus</code> (npm)                                                                                             |
| <b>Độ trưởng thành</b>  | Cao — có trendshift badge, Discord server, CI/CD đầy đủ, changelog, migration guide                                     |
| <b>Hoạt động</b>        | Rất tích cực — AGENTS.md last updated 2026-07-16, changelog và migration docs được duy trì                              |
| <b>Stack</b>            | TypeScript + Node.js                                                                                                    |
| <b>Đặc điểm nổi bật</b> | Self-dogfooding: GitNexus sử dụng chính GitNexus để index codebase của mình, sinh skills và CLAUDE.md/AGENTS.md tự động |

**Ghi chú bước tiếp theo:** Tài liệu này là phần phân tích trung lập — chưa kèm đề xuất áp dụng. Bước tiếp theo (Mode A: đề xuất cải tiến KZTEK / Mode B: giải thích tương tác) phụ thuộc vào lựa chọn của user.